



Slides for P3642R1

Carry-less product: `std::clmul`

Document number:

P3647

Date:

2025-05-27

Audience:

SG22

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:

Jan Schultke <janschultke@gmail.com>

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/clmul-slides.cow



IMPORTANT: This document has custom controls:

-  : go to the next slide
-  : go to previous slide

Carry-less product:

`std::clmul`

P3642R1

Introduction

Intuition: "carry-less" means we use XOR instead of plus.

Regular multiplication	Carry-less multiplication
<pre>x * 0b0110 == (x << 3) * 0 + (x << 2) * 1 + (x << 1) * 1 + (x << 0) * 0</pre>	<pre>clmul(x, 0b0110) == (x << 3) * 0 ^ (x << 2) * 1 ^ (x << 1) * 1 ^ (x << 0) * 0</pre>

- useful for CRC, AES-GCM, parsing, bit manipulation, ...
- widespread hardware support (x86_64, ARM, RISC-V)
- a.k.a. "polynomial multiplication" and "XOR multiplication"

Motivating example

- `clmul(x, -1u)` computes *bitwise parity* (inclusive)
- i.e. for each bit in `x`, */* is 1-bit count to right odd? */ ? 1 : 0*
- can be used to check if character is inside/outside string in parallel

```
abc xxx "foobar" zzz "a"  
0000000001000000100000101 // quotes  
0000000000111111100000011 // clmul(quotes, -1u)  
0000000000111111000000010 // clmul(quotes, -1u) & ~quotes
```

This technique is used to accelerate string parsing in `simdjson`.

Hardware support

Operation	x86_64	ARM	RV64
clmul u64×4 → u128×4 clmul u64×2 → u128×2	vpclmulqdq vpclmulqdq		
clmul u64 → u128	pclmulqdq	pmull+pmull2	clmul+clmulh
clmul u64 → u128	pclmulqdq	pmull+pmull2	clmul+clmulh
clmul u64 → u64		pmull	clmul
clmul u8×8 → u16×8 clmul u8×8 → u8×8		pmull pmul	

Marked rows are integrated in this proposal.

Proposed design

```
template<unsigned-integer T>
T clmul(T x, T y) noexcept;

template<class T>
struct mul_wide_result { // yoinked from P3161R4:
    T low_bits;           // Unified integer overflow arithmetic
    T high_bits;
};

template<unsigned-integer T>
constexpr mul_wide_result<T> clmul_wide(T x, T y) noexcept;
```

- clmul names used because it is most common (Intel, LLVM, RV64, etc.)
- SIMD support could be separate paper

Implementation and wording

Implementation

- naive fallback implementation is trivial
- just need to wrap platform intrinsics when available
- portable support with `@llvm.clmul`
 - could be wrapped in `__builtin_clmul`

Wording

- based on P3161R4, but easy to change
- *see paper*

k thx bye („• ~ •,,)